Diagrama de clases

Magister Víctor Andrade Soto

Definiciones: Clases

Las **clases** representan conceptos o entidades del negocio en la programación orientada a objetos, agrupando objetos que comparten características, condiciones y significado. Para identificarlas en un texto o idea, la estrategia más rápida es buscar los **sustantivos**.

Las clases representan conceptos variados (como *Animal*, *Persona* o *Mensaje*), y su correcta identificación es crucial para evitar tener que rehacer el trabajo de análisis más adelante. A nivel de estructura, una clase se compone de tres elementos principales: **nombre**, **atributos** y **funciones** (cuya visibilidad en el diagrama depende del nivel de detalle requerida).

- Para representar visualmente una clase se usa una **caja dividida en tres zonas** mediante líneas horizontales:
- **Primera zona (superior):** Contiene el nombre de la clase (en *cursiva* si la clase es abstracta).
- **Segunda zona (intermedia):** Lista los atributos de la clase, uno por línea.

visibilidad nombre_atributo : tipo = valor-inicial { propiedades }

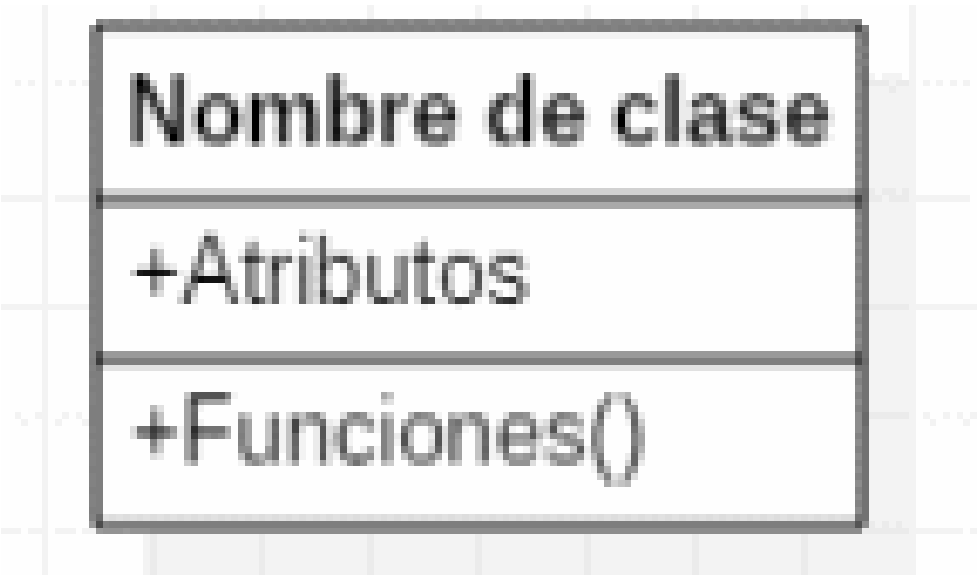
Aunque esta es la forma «oficial» de escribirlas, es común simplificando únicamente poniendo el nombre y el tipo o únicamente el nombre.

La última de las zonas incluye cada una de las funciones que ofrece la clase. De forma parecida a los atributos, sigue el siguiente formato:

visibilidad nombre_funcion { parametros } : tipo-devuelto { propiedades }

De la misma manera que con los atributos, se suele simplificar indicando únicamente el nombre de la función y, en ocasiones, el tipo devuelto.

Una clase



Tanto los atributos como las funciones incluyen al principio de su descripción la visibilidad que tendrá. Esta visibilidad se identifica escribiendo un símbolo y podrá ser:

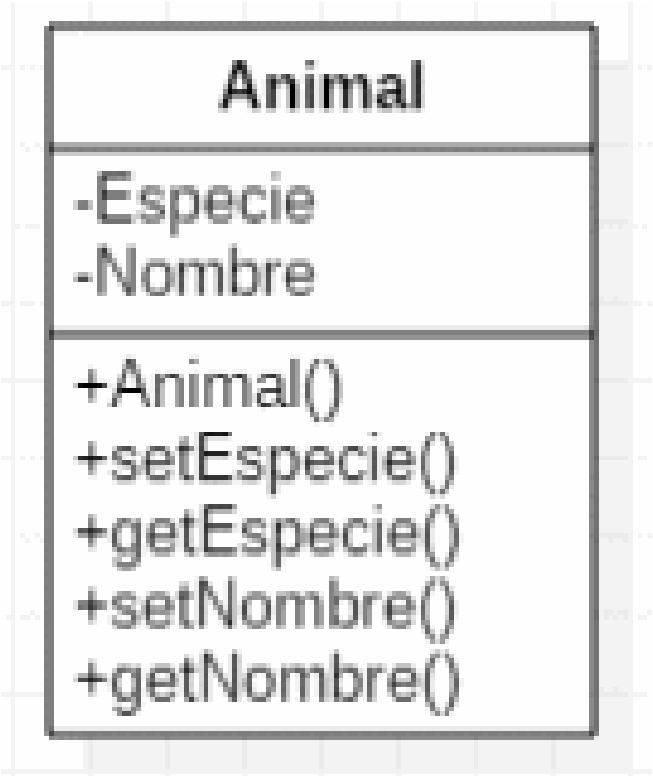
(+) Pública. Representa que se puede acceder al atributo o función desde cualquier lugar de la aplicación.

(-) Privada. Representa que se puede acceder al atributo o función únicamente desde la misma clase.

(#) Protegida. Representa que el atributo o función puede ser accedida únicamente desde la misma clase o desde las clases que hereden de ella (clases derivadas)

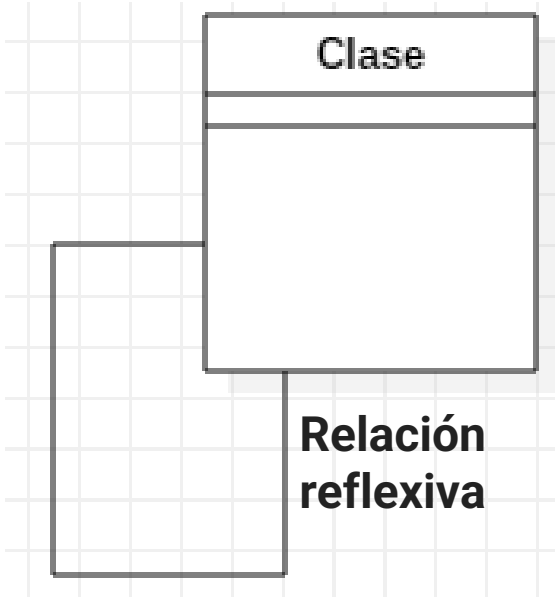
Estos tres tipos de visibilidad son los más comunes. No obstante, pueden incluirse otros en base al lenguaje de programación que se esté usando (no es muy común). Por ejemplo: (/) Derivado o (~) Paquete.

Un ejemplo de clase podría ser el siguiente:



Relaciones

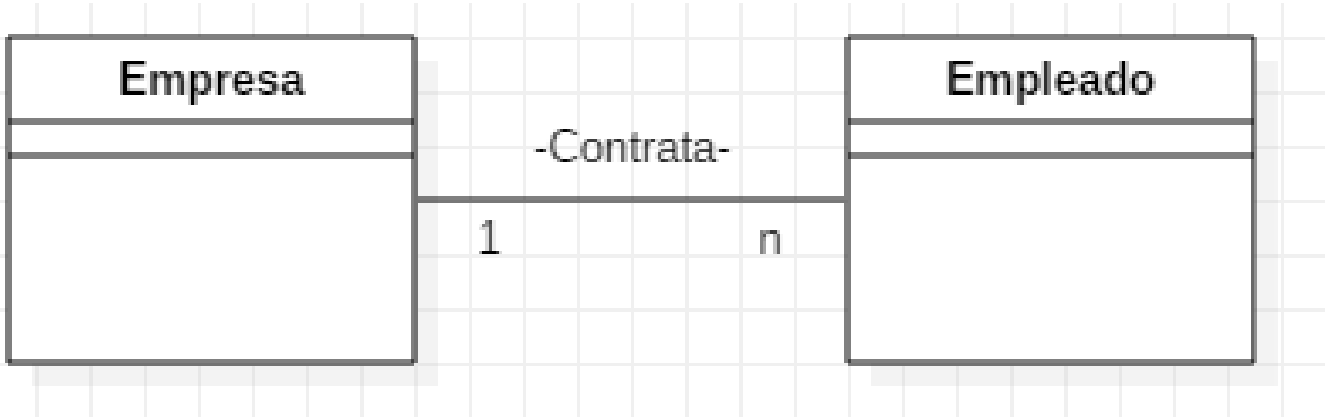
Una relación identifica una dependencia. Esta dependencia puede ser entre dos o más clases (más común) o una clase hacía sí misma (menos común, pero existen), este último tipo de dependencia se denomina dependencia reflexiva. Las relaciones se representan con una línea que une las clases, esta línea variará dependiendo del tipo de relación.



Las relaciones entre clases tienen propiedades opcionales que indican mayor o menor detalle en el diagrama:

Multiplicidad: Indica la cantidad de elementos de una clase que participan en la relación mediante un número, un rango o el símbolo n / * (para un número indeterminado).

Nombre de la asociación: Etiqueta (habitualmente un verbo, como «contrata») que ayuda a describir y comprender la relación entre dos clases.



Tipos de relaciones

Un diagrama de clases incluye los siguientes tipos de relaciones:

Asociación.

Agregación.

Composición.

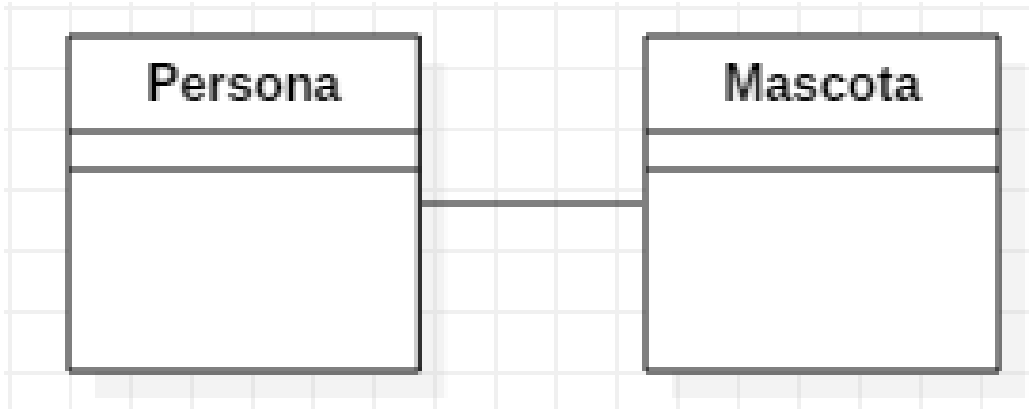
Dependencia.

Herencia.

Asociación

Este tipo de relación es el más común y se utiliza para representar dependencia semántica. Se representa con una simple linea continua que une las clases que están incluidas en la asociación.

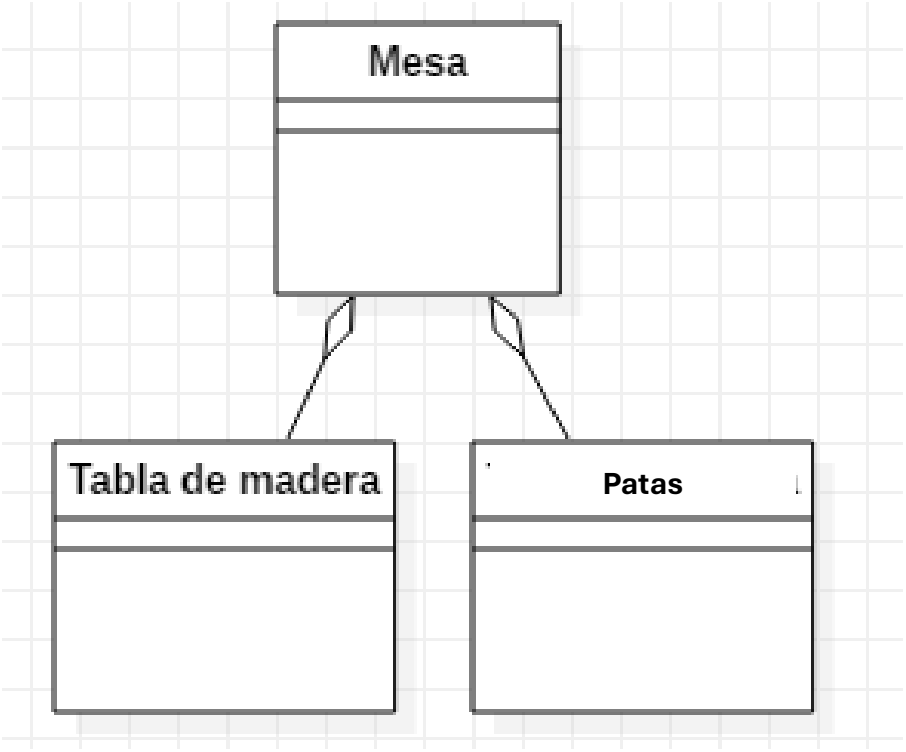
Un ejemplo de asociación podría ser: «Una mascota pertenece a una persona».

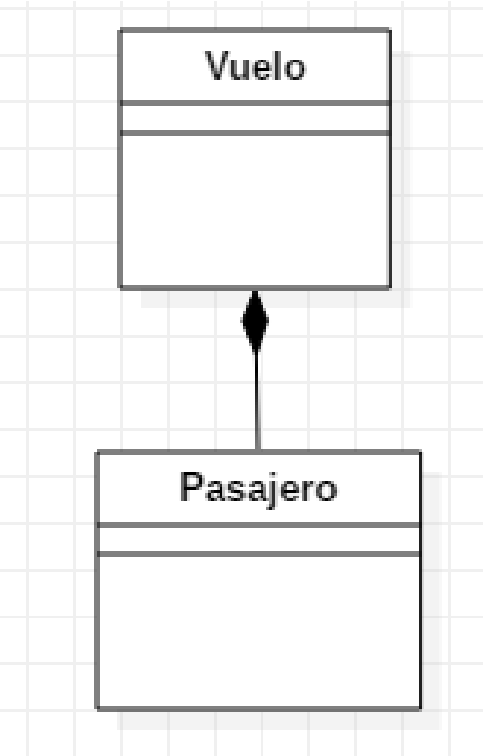


Agregación: Es una representación jerárquica donde un objeto («todo») está compuesto por otros objetos («partes»), pero estas partes pueden existir de forma independiente fuera de esa relación.

Representación gráfica: Se dibuja como una línea con un rombo (vacío/blanco) en el extremo de la clase contenedora (la que agrupa a las partes).

Ejemplo: Una Mesa está compuesta por Tablas de madera y Tornillos; el tornillo es una parte de la mesa, pero puede existir de manera autónoma y formar parte de otros objetos.





Composición:

Composición: Es un tipo de relación jerárquica («todo-parte») más fuerte que la agregación, donde las partes dependen del objeto contenedor para existir.

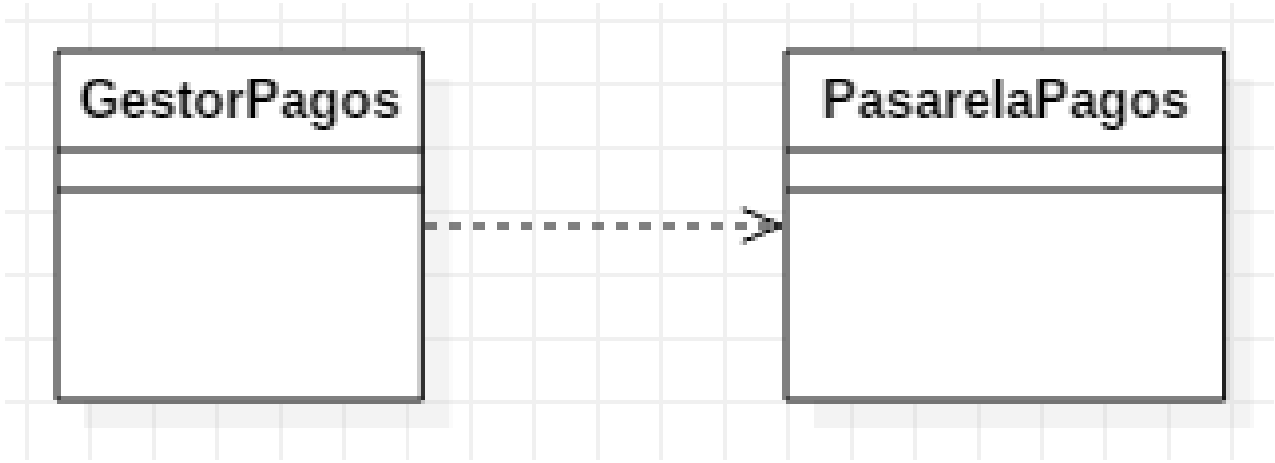
Ciclo de vida compartido: Si la entidad contenedora desaparece, todas sus partes integrantes desaparecen también.

Exclusividad: A diferencia de la agregación, las partes no se comparten entre diferentes objetos contenedores.

Dependencia

Se utiliza este tipo de relación para representar que una clase requiere de otra para ofrecer sus funcionalidades. Es muy sencilla y se representa con una flecha discontinua que va desde la clase que necesita la utilidad de la otra flecha hasta esta misma.

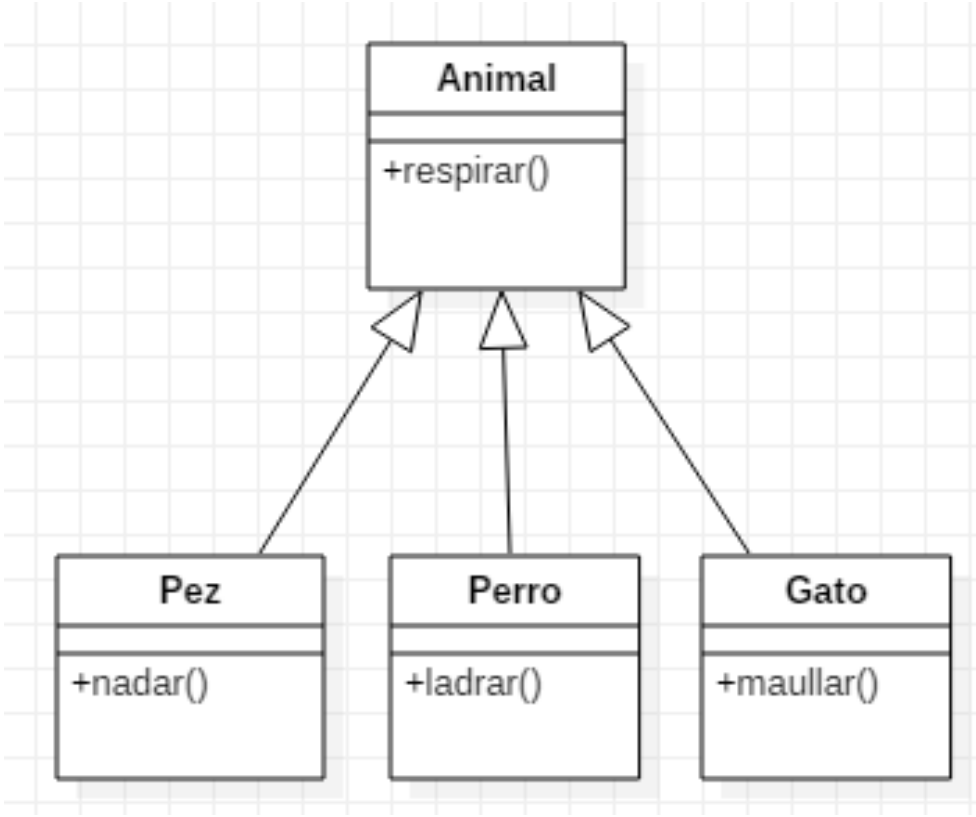
Un ejemplo de esta relación podría ser la siguiente:



Herencia

Otra relación muy común en el diagrama de clases es la herencia. Este tipo de relaciones permiten que una clase (clase hija o subclase) reciba los atributos y métodos de otra clase (clase padre o superclase). Estos atributos y métodos recibidos se suman a los que la clase tiene por sí misma. Se utiliza en relaciones «es un».

Un ejemplo de esta relación podría ser la siguiente: Un pez, un perro y un gato son animales.



Cómo dibujar un diagrama de clases

Base fundamental: Comprender los principios básicos del diseño orientado a objetos es indispensable para elaborar diagramas de clases eficaces.

1. Identificar clases: Consiste en localizar los objetos primarios del sistema, los cuales habitualmente coinciden con los sustantivos dentro del dominio del problema.
2. Distinguir relaciones: Implica determinar las conexiones, puntos en común y abstracciones entre las clases para agruparlas de forma adecuada en el diagrama.

Crea la estructura

Primero, agrega los nombres de clase y vincúlalos con los conectores apropiados, prestando especial atención a la cardinalidad o las herencias. Deja los atributos y funciones para más tarde, una vez que esté la estructura del diagrama resuelta.

Buenas prácticas en la construcción del diagrama de clases

Modularidad: Para evitar incoherencias y desorden al expandir el diagrama, es mejor dividir diagramas grandes en varios más pequeños vinculados entre sí.

Nivel de detalle gradual: Se recomienda crear primero una vista general de alto nivel usando notación simple, y elaborar diagramas detallados por separado según se necesiten.

Claridad visual: Debe evitarse el cruce de líneas en la medida de lo posible para prevenir confusiones al leer el diagrama.

Entidades para una tienda

1.Clientes:

De cada cliente se requiere registrar un identificador único (id), su dirección, un teléfono de contacto y su correo electrónico (email). Un cliente realiza Pedidos dentro del sistema.

2.Productos y Líneas de Producto:

- Del catálogo de Productos, el sistema debe registrar un id, la denominación del producto y el proveedor.
- Cuando un producto es seleccionado para su compra, se gestiona mediante una Línea de Producto (LineaProducto), registrando la cantidad solicitada y el precio aplicable. Cada línea de producto está asociada tanto al producto correspondiente como al Carrito de compra y al Pedido final.

3. Carrito de Compra:

Permite al cliente agrupar los productos antes de realizar el pedido final. Se debe almacenar únicamente la fechaCreación del carrito.

4. Pedidos:

Un Pedido se genera formalmente a partir de los ítems o líneas de producto procesadas. De cada pedido se almacena su id, la fechaRealización, su estado actual y el monto total. Un pedido se vincula directamente con el cliente que lo solicitó.

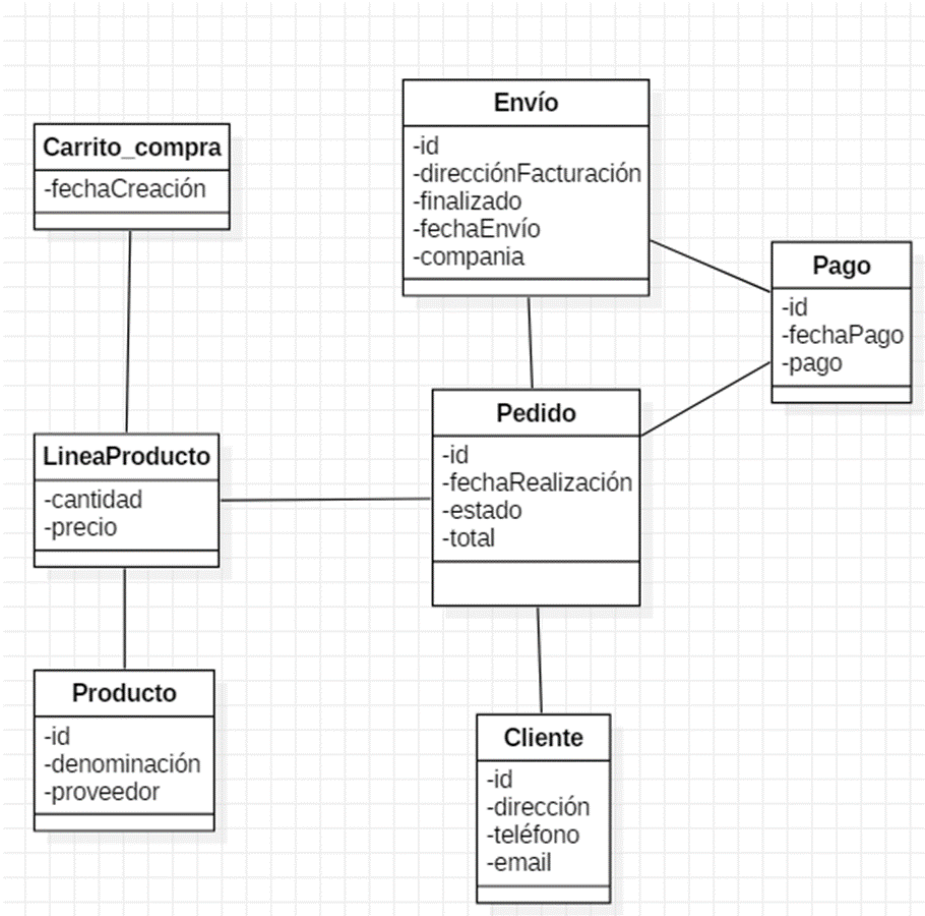
5. Envíos:

Cada pedido puede generar un registro de Envío, el cual almacena un id, la direcciónFacturación, un indicador de si está finalizado, la fechaEnvío y la compañía encargada del transporte.

6. Pagos:

Para concretar el pedido y gestionar el envío, se debe registrar el Pago, el cual contiene su id, la fechaPago y el monto/estado del pago. El pago está relacionado tanto con el Pedido generado como con su respectivo Envío.

Diagrama de clases



Enunciado del Problema: Sistema de Gestión para Clínica Veterinaria

Una clínica veterinaria requiere desarrollar un sistema de información para gestionar la atención de sus pacientes, el historial clínico, las facturaciones y el personal. A continuación se detallan las especificaciones del dominio:

1. Personas y Clientes: Se requiere almacenar información general de las Personas clientes de la clínica, especificando sus datos de contacto: email, dirección y teléfono.
   - Las personas pueden clasificarse en dos tipos:
     - Fisica: Se diferencia por registrar su DNI.
     - Juridica: Se diferencia por registrar su CIF.
2. Una Persona puede ser propietaria de múltiples Animales ($*$), pero cada Animal pertenece a una única Persona (1).

2. Pacientes (Animales) e Historial

De cada Animal se registra su Tipo, Nombre y Edad.

Cada Animal está asociado de forma exclusiva a exactamente un (1) registro Histórico médico.

Un Histórico tiene como atributo identificador RefHistorico y se compone de varios ($*$) ElementoHistorico que detallan eventos pasados.

3. Diagnósticos y Atenciones

Un Animal puede recibir múltiples Diagnóstico a lo largo del tiempo, pero cada Diagnóstico está vinculado a un único Animal (1).

Cada Diagnóstico registra una Fecha y una Descripción.

Cada Diagnóstico es emitido/atendido por un único (1) miembro del Personal, mientras que un miembro del Personal puede realizar múltiples ($*$) diagnósticos.

4. Personal de la Clínica

Del Personal de la clínica se almacena el Nombre, Apellidos y FechaContratacion.

El personal se especializa en dos roles principales:

- Veterinario
- Auxiliar

5. Facturación

Cada Diagnóstico genera exactamente una (1) Factura asociada.

La Factura se identifica mediante su RefFactura.

Una Factura incluye uno o varios ($*$)

ElementoFactura detallados, los cuales contienen la información de Elemento, Precio y Cantidad.

Diagrama de clases clínica veterinaria

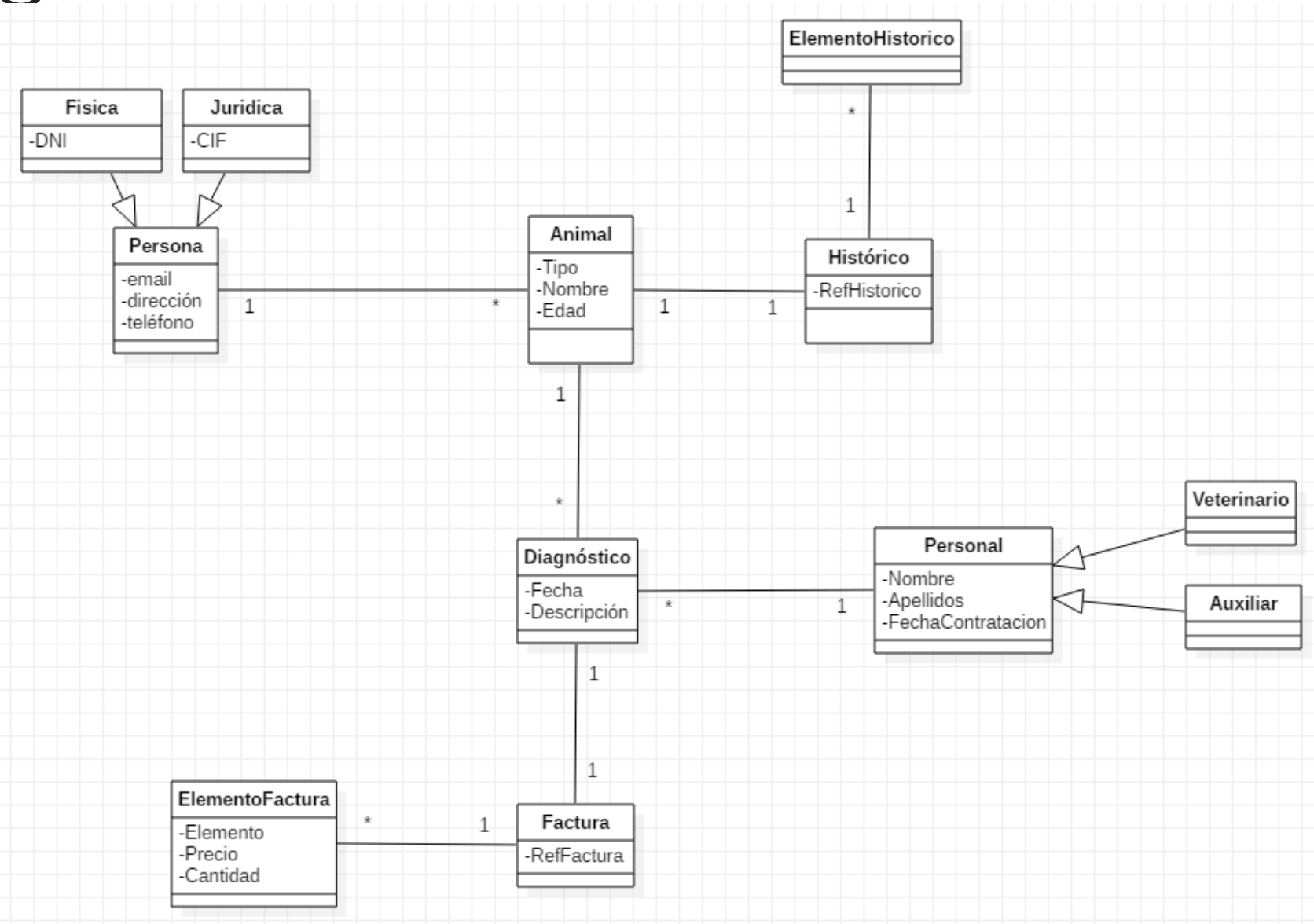


Diagrama de clases zoológico

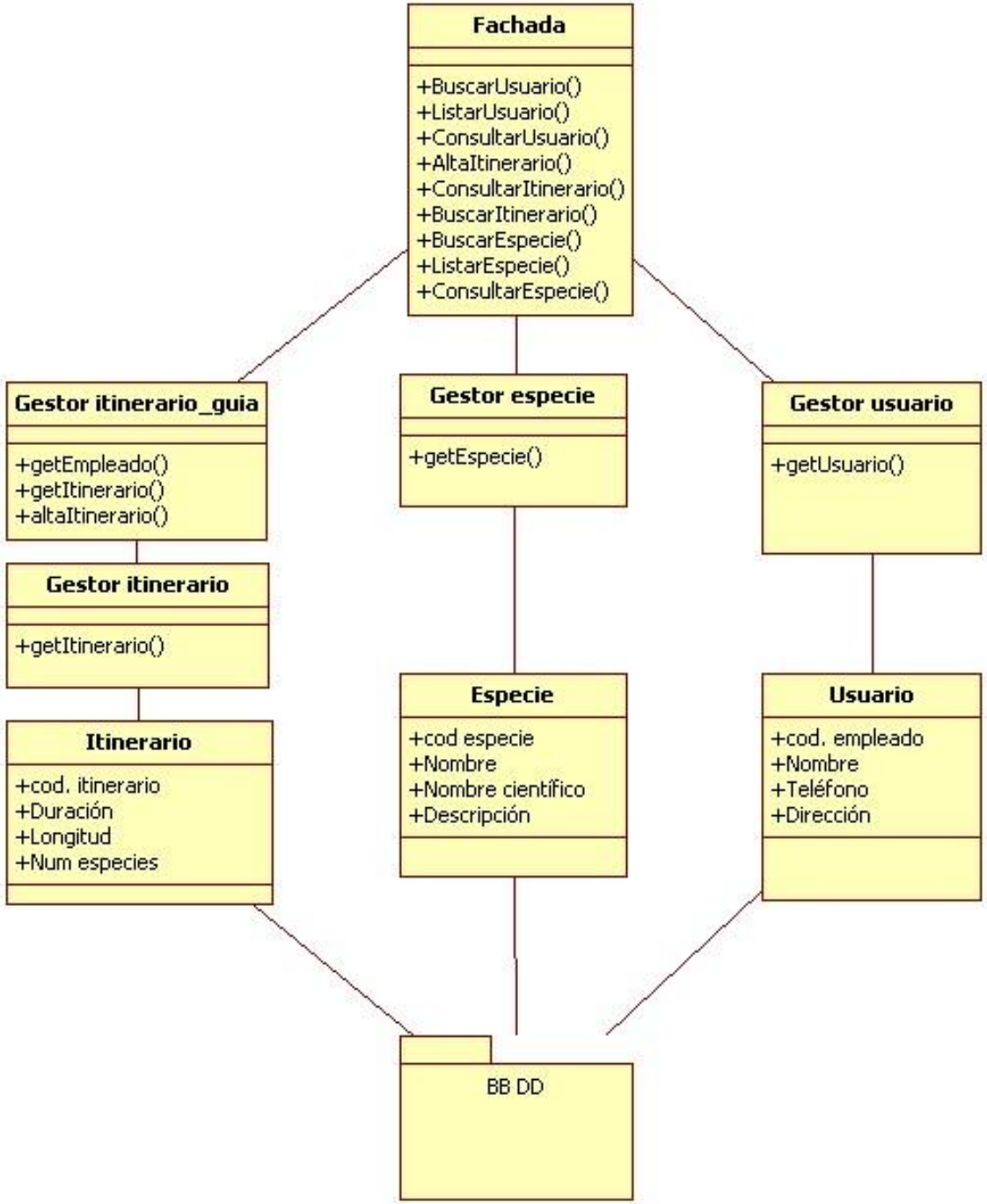


Diagrama de clases gestión de biblioteca

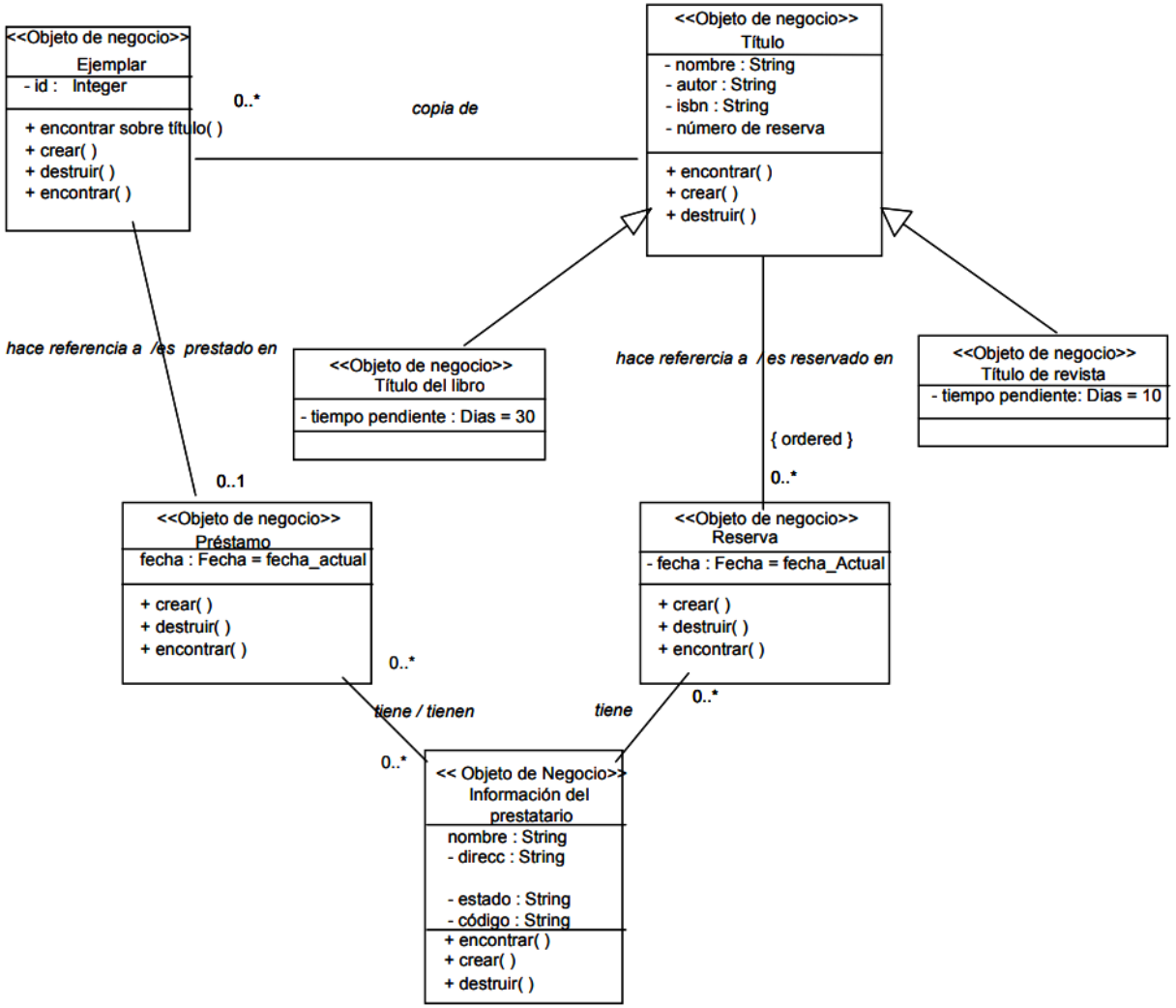
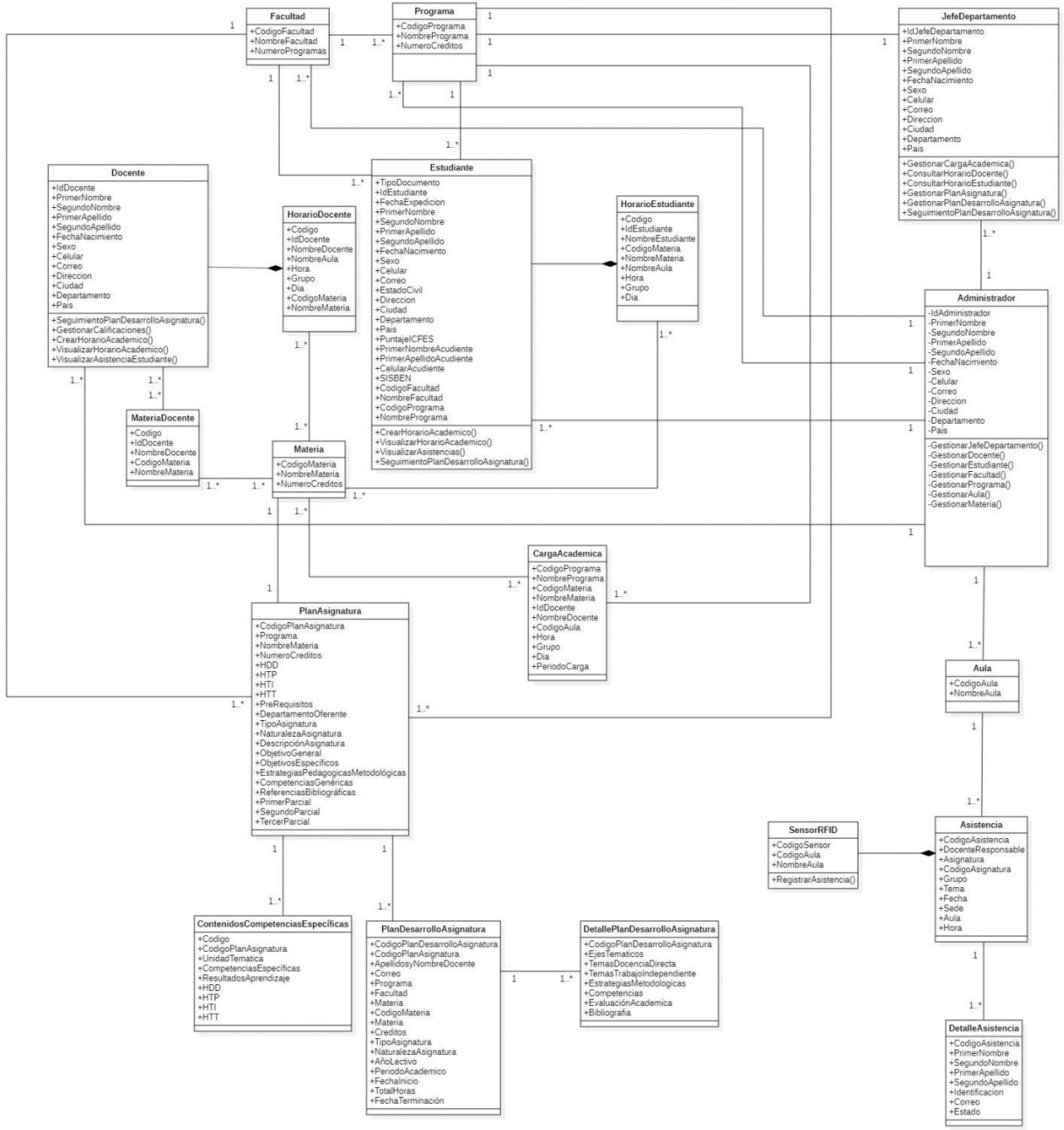


Diagrama de clases centro educativo



Ejemplo de clases para un diagrama de clases de una tienda web

Usuario:

idUsuario: Identificador único del usuario.

nombre: Nombre completo del usuario.

correoElectronico: Dirección de correo electrónico del usuario.

contraseña: Contraseña del usuario.

dirección: Dirección de envío del usuario.

métodoDePago: Método de pago preferido por el usuario.

Producto:

idProducto: Identificador único del producto.

nombre: Nombre del producto.

descripción: Descripción detallada del producto.

precio: Precio del producto.

stock: Cantidad de unidades disponibles en el inventario.

Carrito de Compras:

idCarrito: Identificador único del carrito de compras.

productos: Lista de productos que el usuario ha añadido al carrito.

subtotal: Monto total del carrito antes de aplicar impuestos y descuentos.

impuestos: Monto total de impuestos aplicados al carrito.

Orden de compra:

idOrden: Identificador único de la orden de compra.

productos: Lista de productos comprados en la orden.

subtotal: Monto total de la orden antes de aplicar impuestos y descuentos.

impuestos: Monto total de impuestos aplicados a la orden.

envío: Monto del costo de envío de la orden.

total: Monto total de la orden incluyendo impuestos, descuentos y costo de envío.

Categoría:

idCategoría: Identificador único de la categoría.

nombre: Nombre de la categoría.

Comentarios:

idComentario: Identificador único del comentario.

producto: Identificador del producto al que se refiere el comentario.

usuario: Identificador del usuario que escribió el comentario.

comentario: Contenido del comentario.

fecha: Fecha de creación del comentario.

Espacio Práctico

- Genere el enunciado para los diagramas que no los tienen
- Genere el diagrama para el enunciado de la tienda WEB